

Charles Centeno
219463215
EECS3311-A

STAGE 1 PROJECT DESIGN REPORT – *AI GROCERY COMPANION AGENT*

1.1 Project Description

Project Name: *AI Grocery Companion Agent*

Description:

AI Grocery Companion Agent is a shopping planning system that aims to help users determine what they need to purchase, compare products, and create shopping plans based on their preferences and budget. Its AI functionality allows users to personalize their shopping experience through a centralized interface. Users can describe their needs in natural language, and the agent can easily identify the required items and quantities, retrieve matching products, compare prices and promotions, and generate an organized shopping checklist. It also considers preferred stores, preferred brands, dietary restrictions, allergies, and excluded items when generating recommendations.

What problem does your project solve?

The project addresses issues related to customers' regular shopping routines. In general, shopping requires users to make several decisions: what items they need, how much the purchase will be, which store to buy from, and whether there are other alternatives. While people generally know what they want to buy, they often forget specific products they intend to purchase.

Comparing products takes time as prices, brands, and promotions vary between stores. Users are often obligated to switch between multiple websites to compare options, and accordingly, a cheaper individual product does not necessarily result in a cheaper overall shopping basket. This project reduces that effort by centralizing product searching, price comparison, and shopping optimization in one system.

Who are the target users?

Target users include:

- Grocery shoppers
- Families
- Students
- People purchasing supplies for meals, gatherings, or events.
- People who have dietary restrictions or allergies.
- People completing simple household tasks
- Budget-friendly shoppers

Why is an AI agent appropriate for this problem?

Users describe shopping goals in different ways and may omit details or change their requirements during planning. An AI agent can easily interpret these requests and break each goal into smaller tasks to determine the required items and quantities.

The agent will operate through a multi-step workflow to identify suitable products, retrieve product information, and compare prices and stores for available promotions. Using this information, it will recommend cheaper alternatives and coordinate the creation of an optimized shopping plan based on the user's budget and preferences.

Major Functionalities:

Functionality	Description
1 Natural language request	Users can describe a specific shopping goal. This can include: budget, number of people, name of meal/event/goal, dietary restrictions, constraints and etc.
2 Task decomposition	Break a meal, event, or household goal into required items and estimate suitable quantities and prices.
3 Shopping plan generation	Creates a plan containing item names, categories, required quantities, selected products, brands, stores, prices, and applicable promotions.
4 Item categorization	Organizes items into categories such as household supplies, produce, dairy, seafood, pantry, and etc.
5 Product search	Find matching products and display available brands, prices, and stores accordingly.
6 Price comparison	Compare product prices across stores, using unit prices where package sizes differ.
7 Discount and promotion detection	Identify applicable discounts and offers, including conditions such as loyalty memberships
8 Alternative product recommendations	Suggest suitable substitutes when a product is unavailable, too expensive, or incompatible with the user's preferences.
9 Shopping optimization	Compare shopping plans using shopping strategies: cheapest, fewest stores, one store.
10 Interactive checklists	Allows the user to add, remove, edit, and mark items as purchased while tracking shopping progress.
11 Saved plans and prompts	Allows reopening saved and recent shopping plans, checklists, and reusable prompts.
12 User preference memory	Store preferred shops, brands, dietary restrictions, allergies, and excluded items for future plans.

Which AI/LLM model(s) do you plan to use?

The system will integrate with *Claude Opus 5.5*. The model will interpret the natural-language shopping requests, break goals into required items, recommend suitable alternatives, and coordinate a multi-step planning workflow through application tools. Product search, price calculations, promotion checks, and shopping optimization will be performed by software services. Their result will then be returned to the model to evaluate and optimize a shopping plan based on the user's budget and preferences.

User Interaction:

GUI (Graphical User Interface)

The GUI will include:

1. Home
 - A shopping request chat, recent prompts, saved checklists, and saved plans.
2. Shopping Plan
 - A checklist including: quantities, products, prices, selected stores, promotions, and estimated totals.
 - Optimization button and AI assistant that will anchor plan revisions.
3. Product Search
 - Search results organized by product, brand, price, and store.
4. Price Comparison
 - Comparison of matching products across stores, including unit prices and promotions.
5. User Preferences
 - Settings for preferred stores, brands, dietary restrictions, allergies, and excluded items.

CLI (Command-line interface)

CLI will provide access to the same major functionality through text commands or menus. This includes creating plans, searching for products, comparing prices, inspecting promotions, and selecting strategies.

The CLI will include:

1. Create shopping plan
2. Open checklist
3. Search products
4. Compare Prices
5. View Promotions
6. Manage User Preferences
7. Exit

How will the AI model interact with the rest of the software system?

A ShoppingAgent component will coordinate the model and application tools through the following workflow:

1. Receive the request
 - through GUI or CLI
2. Interpret the goal; extract task, quantities, budget, goals, restrictions, and etc.
3. Retrieve preferences — load relevant user preferences
4. Plan required items — propose a structured list of items and quantities
5. Retrieve products — invoke product-search tools to obtain matching products, prices, stores and promotions
6. Compare and optimize — use application services to calculate the shopping cart total and apply selected strategies

7. Present — display checklist, estimated total, selected stores, and recommendations
8. Save and update — persist the plan when requested and repeat relevant steps when a change of requirement is asked

Involved design patterns:

Pattern		Application
1	MVC	Separation of GUI, shopping data, and logic of the application
2	Strategy	Implement interchangeable optimization of different shopping plans.
3	Observer	Notify GUI when shopping items, purchased status, or calculated totals change.
4	Command	Represent checklist changes, such as adding, removing, replacing, or checking an item.
5	Adapter	Standardize sample catalogues from different store data into a common product and pricing format.

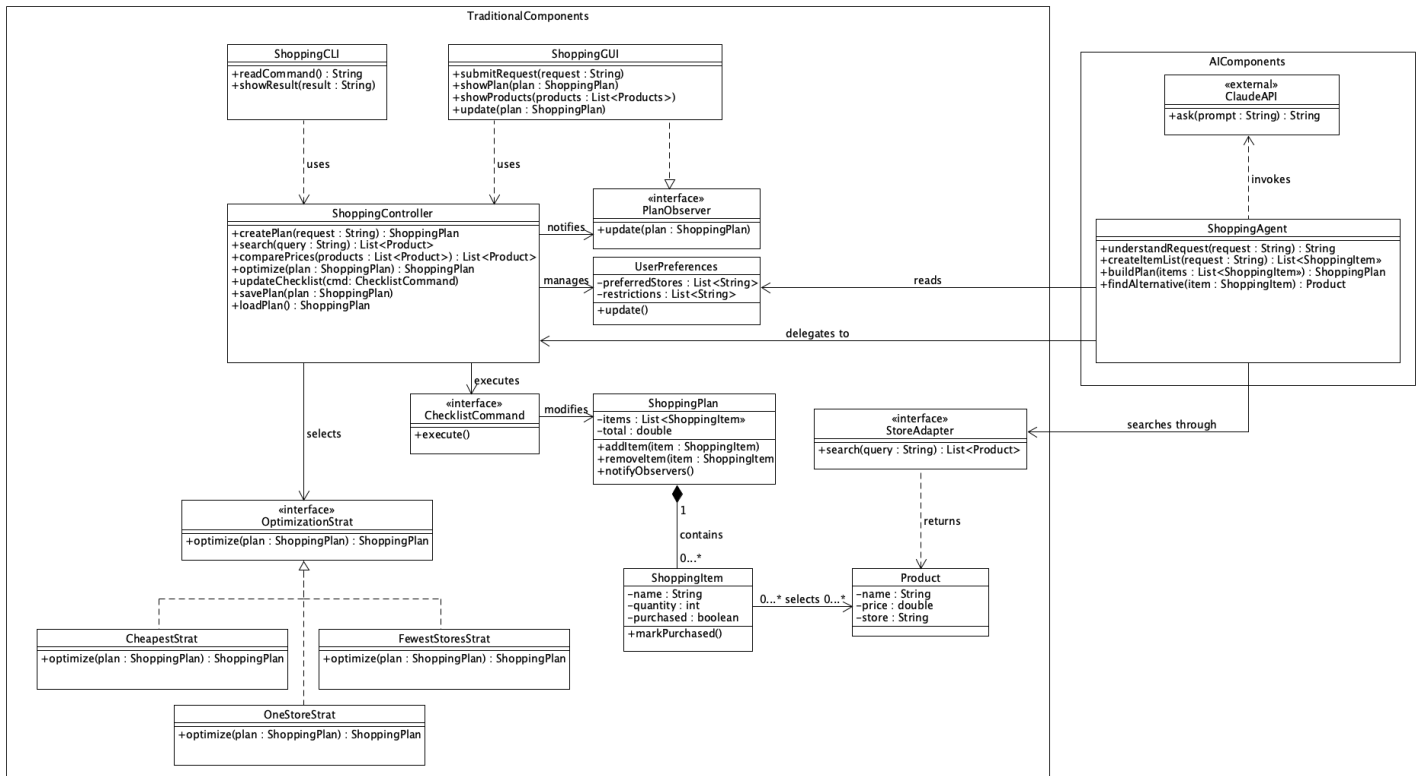
1.2 Feature Specification

ID	Name	Description
F01	Natural Language Request (AI-based)	Allows users to describe a shopping goal in natural language. The user enters their request using the GUI shopping chat. Input: Can be a goal, meal, event, task, budget, restrictions, or preferences. Output: Structured interpretation of shopping needs. Agent interprets the request, extracts constraints and prepares data for planning. If information is unclear, the agent asks for clarification.
F02	Task decomposition (AI-based)	Breaks a general shopping goal into required items and estimated quantities. The generated item list is displayed for review and editing. Input: Shopping goal, number of people, budget, preferences, and restrictions. Output: List of required items and estimated quantities. The agent analyzes the goals, identifies the required items, and estimates suitable quantities. If the request is unclear, the agent asks for additional information.
F03	Shopping plan generation (Hybrid)	Creates a complete shopping plan using the required items, products, prices, stores, and promotions. The user can generate and view the plan through the <i>Shopping Plan</i> screen. Input: Required items, budget, preferences, product data, and optimization strategy. Output: Shopping plan containing products, quantities, stores, prices, promotions, and estimated total. The system retrieves product information, and the agent organizes suitable options into a shopping plan. If products are unavailable or the plan exceeds the budget, alternatives are suggested.
F04	Item categorization (Hybrid)	Organizes shopping items into categories such as produce, dairy, meat, frozen foods, and household supplies. Categories are displayed automatically in the shopping plan and checklist. Input: Item names and information. Output: Categorized shopping list. The system uses predefined categories when possible, while AI may classify unclear items. Items that cannot be categorized are placed under Other category.
F05	Product search (Hybrid)	Searches available product data for products matching a shopping item. The user can search through the <i>Product Search</i> screen or select an item from an existing plan. Input: Product name, brand, store, package size, dietary requirements, or filters. Output: Matching products including brand, price, size, store, and promotion information.

		The agent interprets the search request while product-search services retrieve matching results. If no exact product is found, it suggests similar products.
F06	Price comparison (Deterministic)	Compares similar products across different stores and calculates unit prices according to sizes. The user accesses this feature through the <i>Price Comparison</i> screen. Input: Product prices, sizes, quantities, and stores. Output: Price comparison, unit prices, and estimated savings. The system normalizes product sizes, calculates unit prices, and ranks available options. The system identifies and excludes products with missing or incompatible information from direct comparison.
F07	Discount and promotion detection (Hybrid)	Identifies available discounts, promotions, and loyalty offers for products. Promotions are displayed beside products and within the shopping plan. Input: Product, store, price, quantity, and promotions. Output: Applicable promotions and discounted prices. The system checks promotion requirements and applies eligible discounts. Expired or conditional promotions are clearly identified instead of automatically applied.
F08	Alternative product recommendations (Hybrid)	Recommends substitute products when an item is unavailable, too expensive, or unsuitable for the user's preferences. The user can select <i>Find Alternative</i> from the shopping plan. Input: Current product, preferences, restrictions, price, and available alternatives. Output: Ranked list of suitable substitute products. The system retrieves alternatives and the agent evaluates which options best match the user's needs. If no suitable substitute is available, the item remains unresolved.
F09	Shopping optimization (Hybrid)	Optimizes the shopping plan using strategies such as <u>Cheapest</u>, <u>Fewest Stores</u>, or <u>One Store</u>. The user selects the desired strategy from the <i>Shopping Plan</i> screen. Input: Required items, products, prices, stores, promotions, budget, and strategy. Output: Optimized shopping plan and estimated total. The selected optimization algorithm compares product and store combinations, while the agent evaluates the final plan against the user's needs. If all requirements cannot be satisfied, the closest available option is shown.
F10	Interactive checklists (Deterministic)	Allows users to manage and track shopping items through an interactive checklist. Users can add, remove, edit, replace, or mark items as purchased. Input: Shopping plan and user checklist actions.

		Output: Updated checklist, shopping progress, and estimated total. The system updates the checklist and recalculates totals as changes are made. Invalid quantities or unavailable replacements are rejected, and an error message is displayed.
F11	Saved plans and prompts (Hybrid)	Allows users to save shopping plans, checklists, and reusable shopping requests. Saved items can be accessed again through the Home screen. Input: Shopping plan, checklist, request, and optional plan name. Output: Stored shopping plan or reusable request. The system saves the information and allows it to be reopened and updated later. If product prices or availability have changed, the system informs the user and allows the data to be refreshed.
F12	User preference memory (Hybrid)	Stores reusable shopping preferences for future shopping plans. Users manage these preferences through the <i>User Preferences</i> screen. Input: Preferred stores, brands, dietary restrictions, allergies, and excluded items. Output: Saved user preference profile. The system stores the preferences, and the agent retrieves them when generating future recommendations. If a current request conflicts with a saved preference, the user is notified and can override it.

2.1 Class Diagram



Made using UMLet

Patterns Used:

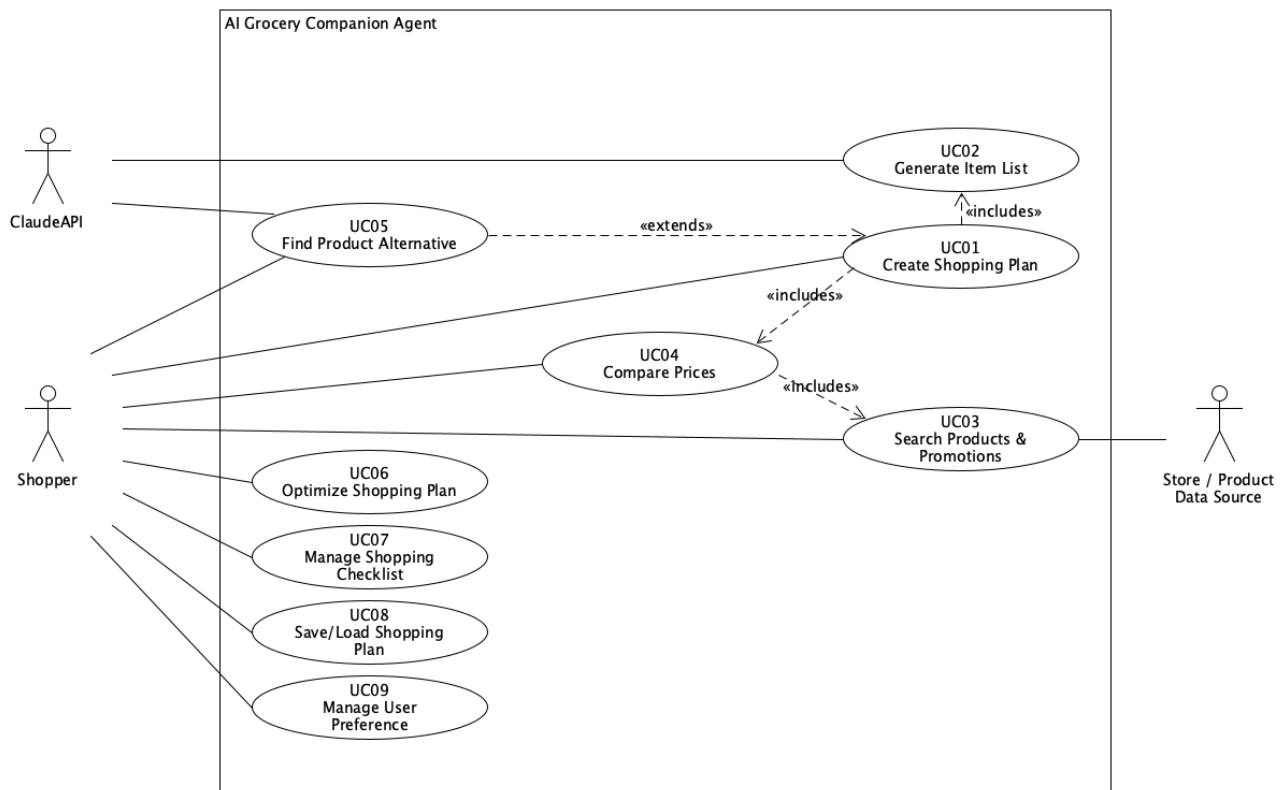
Pattern	Explanation
1 MVC	Problem: The GUI, program logic, data separation Classes: ShoppingGUI, ShoppingCLI, ShoppingController, ShoppingPlan, ShoppingItem, Product. Roles: GUI/CLI handle user interaction, the controller handles actions, and the model classes store shopping data. Why: It keeps the program organized and easier to change. Why not: User interface code and shopping logic could become mixed together.
2 Strategy	Problem: The user can optimize plans in different ways. Classes: OptimizationStrat, CheapestStrat, FewestStoresStrat, OneStoreStrat. Roles: Each strategy contains different optimization methods. Why: Easy switching between strategies. Why not: Would require more coding, such as if/else statements.

3	Observer	Problem: GUI needs to update when the shopping plan changes. Classes: <code>ShoppingPlan</code>, <code>PlanObserver</code>, <code>ShoppingGUI</code>. Roles: <code>ShoppingPlan</code> sends notifications, and <code>ShoppingGUI</code> updates itself. Why: GUI stays synchronized within the system. Why not: The program would need to manually refresh GUI.
4	Command	Problem: Checklist actions like adding or removing items should be handled consistently. Classes: <code>ChecklistCommand</code>, <code>AddItemCommand</code>, <code>RemoveItemCommand</code>, <code>ShoppingController</code>. Roles: Each command represents one checklist action, and the controller executes it. Why: Checklist actions are easier to organize and extend. Why not: The controller would contain all checklist logic directly.
5	Adapter	Problem: Different stores may have different product data formats. Classes: <code>StoreAdapter</code>, <code>Product</code>. Roles: Each adapter converts store data into the same <code>Product</code> format. Why: The rest of the system can treat all store data the same way. Why not: The agent would need separate code for every store.

2.2 Use Case Diagram

List of Actors

1. Claude API
2. Shopper
3. Store/Product Data Source



Made using UMLet

Use Cases

ID	Name	Description
UC01	Create shopping plan	Actor: Shopper Goal: Create a complete shopping plan from a natural-language request. Precondition: The application is running. Trigger: The shopper submits a request. Main Success Scenario: The system interprets the request, generates the required item list, searches for products, compares prices, and displays a shopping plan. Alternative / Exception Flow: If information is missing or unclear, the system asks the shopper for clarification. Post Condition: A shopping plan is created and displayed. Related Feature: F01, F03

UC02 Generate item list	Actor: Claude API Goal: Identify items needed for the shopper's goal. Precondition: A request should exists. Trigger: <u>Create Shopping Plan</u> includes this use case. Main Success Scenario: The AI analyzes the request, identifies the required items and quantities, and creates the shopping item list. Alternative / Exception Flow: If the request is unclear, more information is requested. Post Condition: A list of required shopping items is developed. Related Feature: F02, F04
UC03 Search products & promotions	Actor: Shopper, Product Data Source Goal: Find products that match the item and find available promotions. Precondition: A product request or shopping item exists. Trigger: The shopper searches for a product or the system needs product information. Main Success Scenario: The system searches available store data and displays matching products, prices, stores, and promotions. Alternative / Exception Flow: If no matching product is found, the system informs the shopper. Post Condition: Matching product options are available. Related Feature: F05, F07
UC04 Compare prices	Actor: Shopper Goal: Compare similar products across different stores. Precondition: Product options are available. Trigger: The shopper selects <u>Compare Prices</u> or a shopping plan requires a comparison. Main Success Scenario: The system compares available product prices and displays the results. Alternative / Exception Flow: Products with missing price information are excluded or marked unavailable. Post Condition: The shopper can identify the better-value option. Related Feature: F06
UC05 Find product alternative	Actor: Shopper, Claude API Goal: Find a suitable replacement for an unavailable, unsuitable, or expensive product. Precondition: A shopping item or selected product exists. Trigger: The shopper requests an alternative or the current product cannot be used. Main Success Scenario: The system searches for alternatives and the AI

		recommends a suitable replacement. Alternative / Exception Flow: If no suitable alternative is found, the shopper is informed. Post Condition: An alternative product may be selected. Related Feature: F08
UC06	Optimize shopping plan	Actor: Shopper Goal: Optimize the shopping plan using Cheapest, Fewest Stores, or One Store. Precondition: A shopping plan exists. Trigger: The shopper selects an optimization strategy. Main Success Scenario: The system applies the selected strategy and displays the optimized plan. Alternative / Exception Flow: If the selected strategy cannot satisfy all items, the closest available plan is displayed. Post Condition: The shopping plan reflects the selected optimization strategy. Related Feature: F09
UC07	Manage shopping checklist	Actor: Shopper Goal: Add, remove, or mark shopping items as purchased. Precondition: A shopping plan exists. Trigger: The shopper performs an action on the checklist. Main Success Scenario: The selected checklist action is executed, and the shopping plan is updated. Alternative / Exception Flow: Invalid actions are rejected, and an error message is displayed. Post Condition: The checklist reflects the shopper's latest changes. Related Feature: F10
UC08	Save / Load shopping plan	Actor: Shopper Goal: Save a shopping plan for later use or reopen a saved plan. Precondition: A plan exists when saving, or a saved plan exists when loading. Trigger: The shopper selects Save Plan or Load Plan. Main Success Scenario: The system saves the current plan or loads a previously stored plan. Alternative / Exception Flow: If the operation fails, the shopper is informed. Post Condition: The plan is saved or loaded successfully. Related Feature: F11
UC09	Manage user preferences	Actor: Shopper Goal: Store or modify preferred stores and shopping restrictions.

Precondition: The application is running.

Trigger: The shopper opens User Preferences.

Main Success Scenario: The shopper changes their preferences and the system stores the updated information.

Alternative / Exception Flow: Invalid preference information is rejected or corrected.

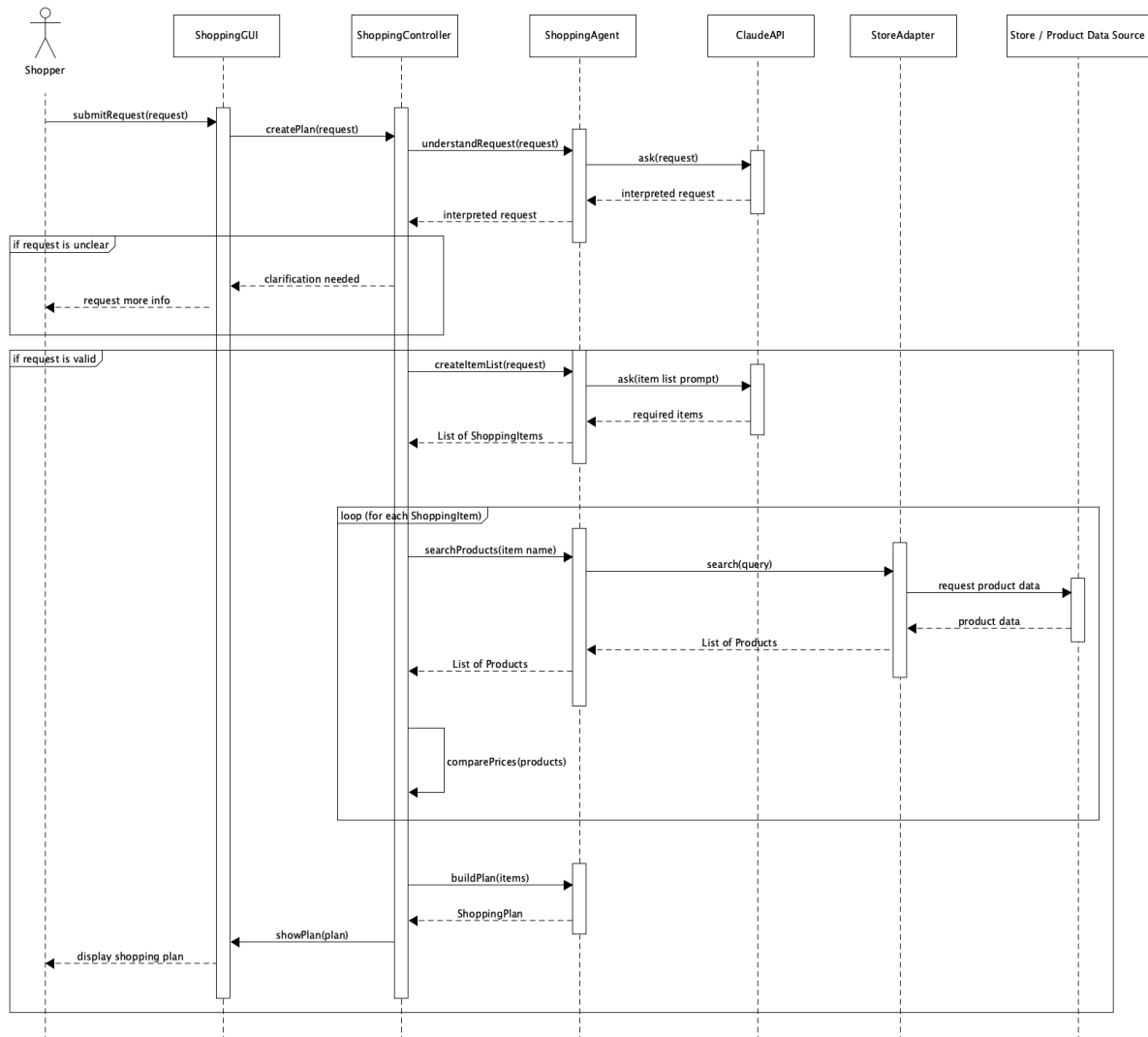
Post Condition: Updated preferences are available for future shopping plans.

Related Feature: F12

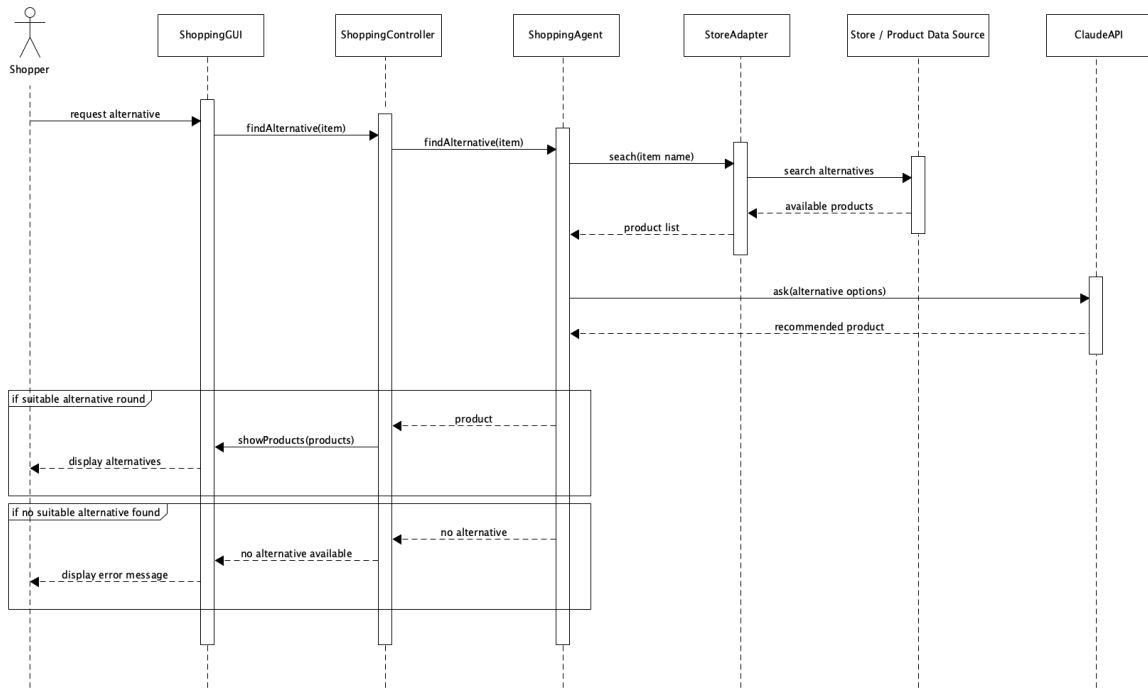
2.3 Sequence Diagrams

Sequence Diagrams

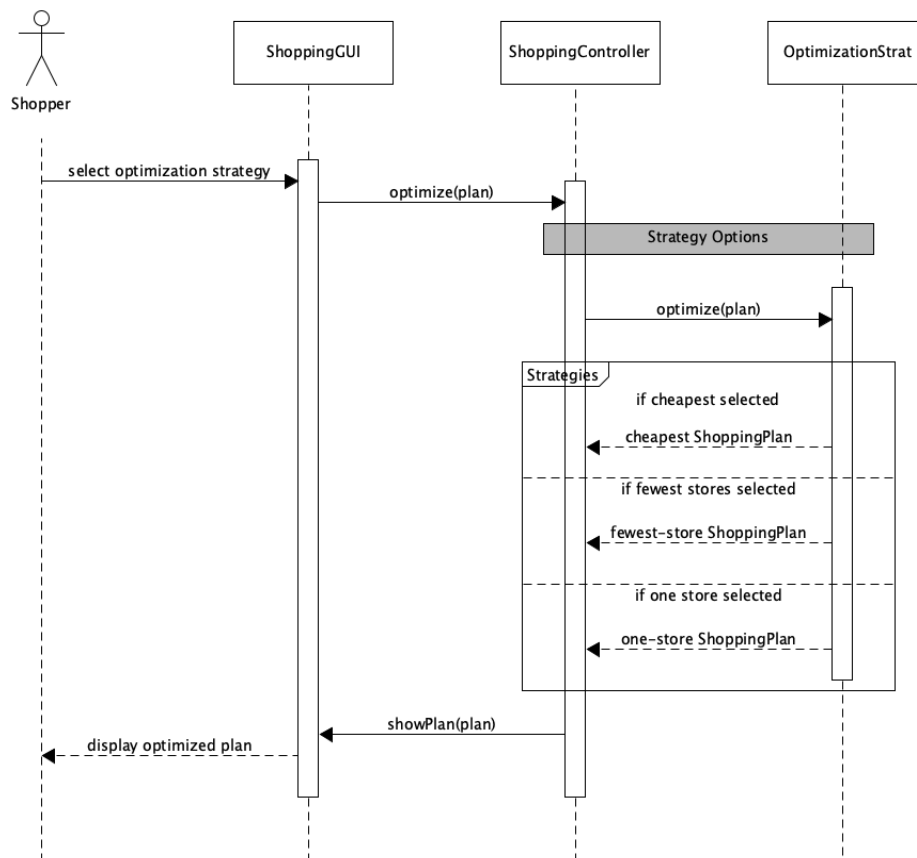
- SD01 – Generate Shopping Plan
- SD02 – Find Product Alternative
- SD03 – Optimize Shopping Plan
- SD04 – Manage Shopping Checklist
- SD05 - Save/Load Shopping Plan and Manage User Preferences



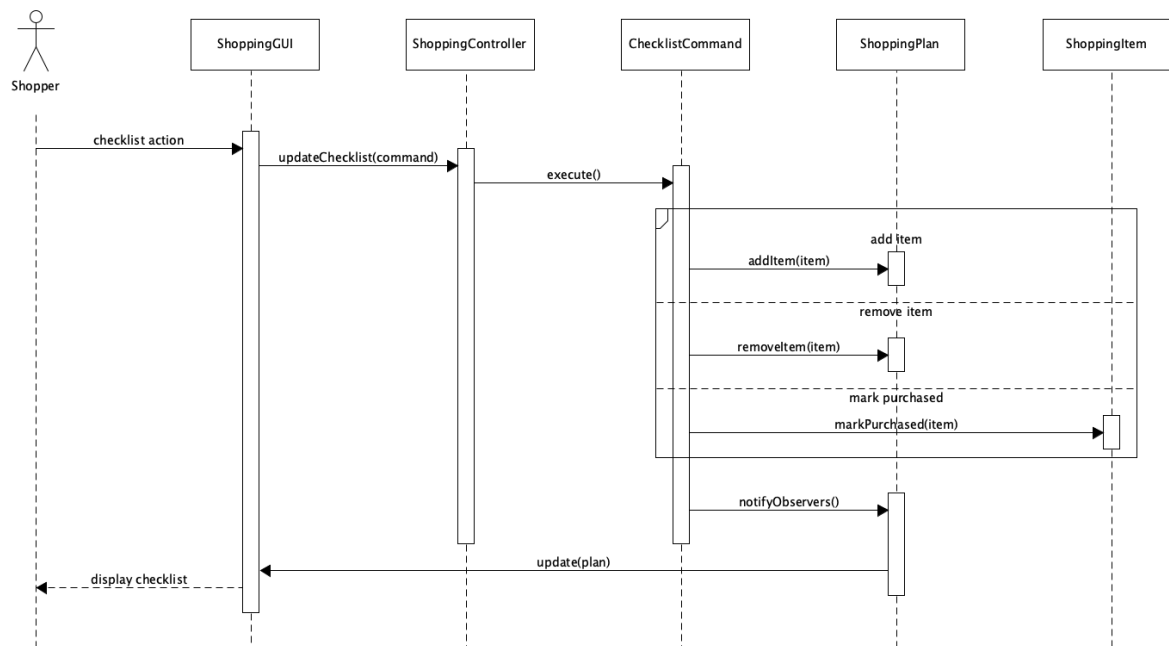
SD01 - Generate Shopping Plan



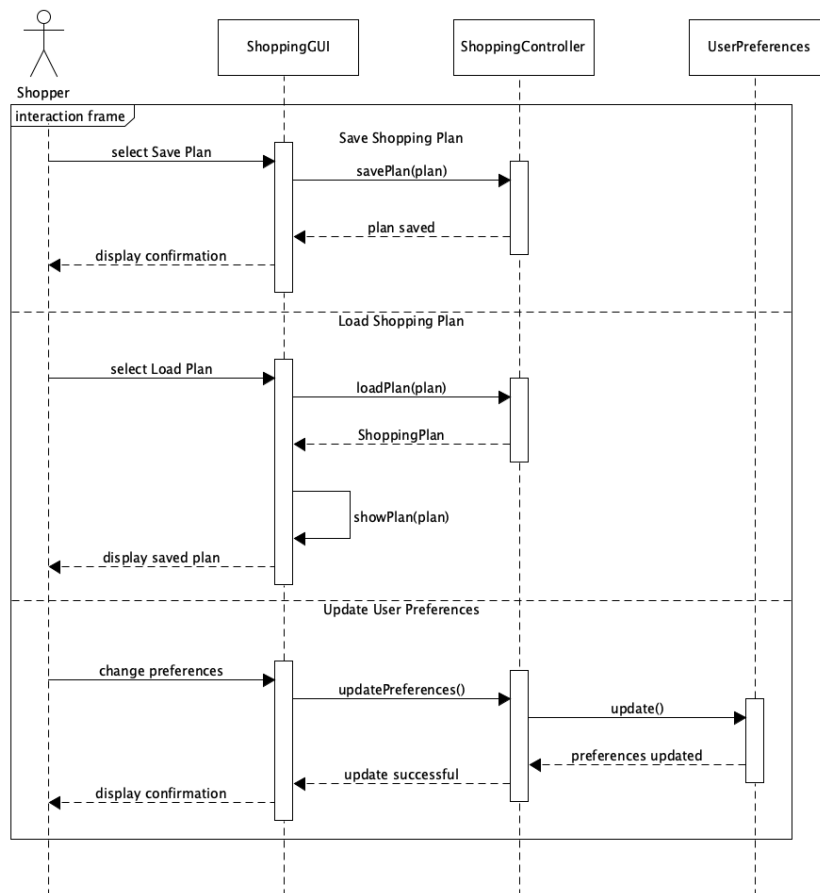
SD02 - Find Alternative



SD03 - Optimize Shopping Plan



SD04 - Manage Shopping Checklist



SD05 - Save/Load Shopping Plan and Manage Preferences

3. Feature-to-Design Traceability

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Natural language request	AI-based	UC01 Create Shopping Plan	ShoppingGUI, ShoppingController, ShoppingAgent, ClaudeAPI	<code>submitRequest(), createPlan(), understandRequest(), ask()</code>	SD01	MVC
F02	Task decomposition	AI-based	UC02 Generate Item List	ShoppingController, ShoppingAgent, ClaudeAPI, ShoppingItem	<code>createItemList(), ask()</code>	SD01	MVC
F03	Shopping plan generation	Hybrid	UC01 Create Shopping Plan	ShoppingGUI, ShoppingController, ShoppingAgent, ShoppingPlan, ShoppingItem, Product, StoreAdapter	<code>createPlan(), createItemList(), search(), comparePrices(), buildPlan(), showPlan()</code>	SD01	MVC, Adapter
F04	Item categorization	Hybrid	UC02 Generate Item List	ShoppingAgent, ShoppingItem, ShoppingPlan	<code>createItemList(); categorizeItem()</code>	SD01 (partial)	MVC
F05	Product search	Hybrid	UC03 Search Products & Promotions	ShoppingGUI, ShoppingController, ShoppingAgent, StoreAdapter, Product	<code>search(), showProducts()</code>	SD01, SD02 (partial)	MVC, Adapter
F06	Price comparison	Deterministic	UC04 Compare Prices	ShoppingController, Product	<code>comparePrices()</code>	SD01 (partial)	MVC
F07	Discount and promotion detection	Hybrid	UC03 Search Products & Promotions	ShoppingController, StoreAdapter, Product	<code>search(); checkPromotions(), calculateDiscount()</code>	SD01 (partial)	Adapter, MVC
F08	Alternative product recommendations	Hybrid	UC05 Find Product Alternative	ShoppingGUI, ShoppingController, ShoppingAgent, StoreAdapter, ClaudeAPI, Product	<code>findAlternative(), search(), ask(), showProducts()</code>	SD02	MVC, Adapter
F09	Shopping optimization	Hybrid	UC06 Optimize Shopping Plan	ShoppingGUI, ShoppingController, OptimizationStrat, CheapestStrat, FewestStoresStrat, OneStoreStrat, ShoppingPlan	<code>optimize(), showPlan()</code>	SD03	Strategy, MVC
F10	Interactive checklists	Deterministic	UC07 Manage Shopping Checklist	ShoppingGUI, ShoppingController, ChecklistCommand, ShoppingPlan, ShoppingItem, PlanObserver	<code>updateChecklist(), execute(), addItem(), removeItem(), markPurchased(), notifyObservers(), update()</code>	SD04	Command, Observer, MVC
F11	Saved plans and prompts	Hybrid	UC08 Save/Load Shopping Plan	ShoppingGUI, ShoppingController, ShoppingPlan	<code>savePlan(), loadPlan(); savePrompt(), loadPrompt()</code>	SD05 (partial)	MVC; Repository/DAO (proposed)
F12	User preference memory	Hybrid	UC09 Manage User Preferences	ShoppingGUI, ShoppingController, UserPreferences, ShoppingAgent	<code>updatePreferences(), update()</code>	SD05	MVC

4. Feature Explanations

F01 — Natural Language Request

Related Use Case: UC01 — Create Shopping Plan

Related Sequence Diagram: SD01 — Create Shopping Plan

Classes Involved:

- ShoppingGUI — collects the shopper's natural-language request.
- ShoppingCLI — provides an alternative command-line interface for entering requests.
- ShoppingController — receives the request and coordinates the shopping plan generation process.
- ShoppingAgent — interprets the shopping request and identifies the shopper's needs.
- ClaudeAPI — processes the natural-language request and returns an interpretation.

Important Methods:

- `ShoppingGUI.submitRequest(request: String)`
- `ShoppingController.createPlan(request: String): ShoppingPlan`
- `ShoppingAgent.understandRequest(request: String): String`
- `ClaudeAPI.ask(prompt: String): String`

Execution

When the shopper enters a request in the GUI, `submitRequest()` sends it to `ShoppingController.createPlan()`. The controller then uses `ShoppingAgent.understandRequest()` to understand what the shopper is looking for. The shopping agent sends the request to `ClaudeAPI.ask()`, which identifies the shopper's goal and important requirements. If some information is missing or unclear, the system asks the shopper for more details before creating the shopping plan.

F02 — Task Decomposition

Related Use Case: UC02 — Generate Item List

Related Sequence Diagram: SD01 — Create Shopping Plan

Classes Involved

- ShoppingController — Coordinates item list generation after interpreting the request.
- ShoppingAgent — Breaks the shopping goal into required items and estimates quantities.
- ClaudeAPI — Assists in identifying items needed to complete the shopping goal.
- ShoppingItem — Represents individual items within the generated shopping list.

Important Methods

- `ShoppingAgent.createItemList(request: String): List<ShoppingItem>`
- `ClaudeAPI.ask(prompt: String): String`

Execution

After understanding the shopper's request, `ShoppingController` calls `ShoppingAgent.createItemList()` to create a list of needed items. The shopping agent uses `ClaudeAPI.ask()` to identify the required items and their quantities based on the shopper's goal. The items are then stored as `ShoppingItem` objects and returned to the controller to search for products and create the shopping plan.

F03 — Shopping Plan Generation

Related Use Case: UC01 — Create Shopping Plan

Related Sequence Diagram: SD01 — Create Shopping Plan

Classes Involved

- ShoppingGUI — Displays the generated shopping plan.
- ShoppingController — Coordinates the shopping plan generation process.
- ShoppingAgent — Generates the required item list and constructs the shopping plan.
- StoreAdapter — Retrieves matching products from available store data sources.

- ShoppingPlan — Represents the generated shopping plan containing shopping items and the estimated total.
- ShoppingItem — Represents individual items and their required quantities.
- Product — Contains product information, including names, prices, and stores.

Important Methods

- `ShoppingController.createPlan(request: String): ShoppingPlan`
- `ShoppingAgent.createItemList(request: String): List<ShoppingItem>`
- `StoreAdapter.search(query: String): List<Product>`
- `ShoppingController.comparePrices(products: List<Product>): List<Product>`
- `ShoppingAgent.buildPlan(items: List<ShoppingItem>): ShoppingPlan`
- `ShoppingGUI.showPlan(plan: ShoppingPlan)`

Execution

When the shopper requests a shopping plan, `ShoppingController.createPlan()` manages the process. The controller uses `ShoppingAgent` to understand the request and create a list of needed items. For each item, `StoreAdapter.search()` finds matching products from different stores, while `comparePrices()` compares their prices. After collecting the product information, `ShoppingAgent.buildPlan()` creates a `ShoppingPlan` containing the selected items and products. The completed shopping plan is then returned to the controller and displayed to the shopper using `ShoppingGUI.showPlan()`.

F04 — Item Categorization

Related Use Case: UC02 — Generate Item List

Related Sequence Diagram: SD01 — Create Shopping Plan (Partial)

Classes Involved

- ShoppingAgent — Generates the shopping item list and is intended to organize items into appropriate categories.
- ClaudeAPI — Assists in interpreting item information and determining appropriate classifications.
- ShoppingItem — Represents individual items within the shopping plan.
- ShoppingPlan — Contains the generated shopping items.

Important Methods

- `ShoppingAgent.createItemList(request: String): List<ShoppingItem>`
- `ClaudeAPI.ask(prompt: String): String`
- `ShoppingAgent.buildPlan(items: List<ShoppingItem>): ShoppingPlan`

Execution

When creating the item list, `ShoppingAgent.createItemlList()` uses `ClaudeAPI` to identify the items needed by the shopper. The items are intended to be grouped into categories such as produce, dairy, meat, and household supplies. These categorized items are then included in the `ShoppingPlan`.

F05 — Product Search

Related Use Case: UC03 — Search Products & Promotions

Related Sequence Diagrams: SD01 — Create Shopping Plan, SD02 — Find Product Alternative

Classes Involved

- ShoppingGUI — Displays available product information.
- ShoppingController — Handles product search requests and coordinates product retrieval.
- ShoppingAgent — Requests product searches during plan generation or alternative product recommendations.
- StoreAdapter — Provides a standardized interface for retrieving product information.
- Product — Represents individual products containing names, prices, and stores.

Important Methods

- `ShoppingController.search(query: String): List<Product>`

- `StoreAdapter.search(query: String): List<Product>`
- `ShoppingGUI.showProducts(products: List<Product>)`

Execution

When the system needs product information, it uses `StoreAdapter` to search for products from different stores. `ShoppingAgent` searches for matching products for each item on the shopping list. The adapter retrieves product details, such as names, prices, and stores, and returns them as `Product` objects. The controller then uses this information to compare prices, create shopping plans, or display products through `ShoppingGUI.showProducts()`.

F06 — Price Comparison

Related Use Case: UC04 — Compare Prices

Related Sequence Diagram: SD01 — Create Shopping Plan

Classes Involved

- `ShoppingController` — Compares retrieved products based on their prices.
- `StoreAdapter` — Retrieves product information from available store data sources.
- `Product` — Contains product prices and store information.
- `ShoppingGUI` — Displays product information and comparison results.

Important Methods

- `StoreAdapter.search(query: String): List<Product>`
- `ShoppingController.comparePrices(products: List<Product>): List<Product>`
- `ShoppingGUI.showProducts(products: List<Product>)`

Execution

After finding matching products using `StoreAdapter.search()`, the product information is sent to the `ShoppingController`. The controller then uses `comparePrices()` to compare product prices from different stores. The results help the system choose products for the shopping plan or display price comparisons to the shopper through the GUI.

F07 — Discount and Promotion Detection

Related Use Case: UC03 — Search Products & Promotions

Related Sequence Diagram: SD01 — Create Shopping Plan (Partial)

Classes Involved

- `StoreAdapter` — Retrieves product information from store or product data sources.
- `Product` — Represents product information retrieved from stores.
- `ShoppingController` — Coordinates product retrieval and price comparisons.
- `ShoppingPlan` — Represents the shopping plan containing selected products and total cost.
- `ShoppingGUI` — Displays product information to the shopper.

Important Methods

- `StoreAdapter.search(query: String): List<Product>`
- `ShoppingController.comparePrices(products: List<Product>): List<Product>`
- `ShoppingGUI.showProducts(products: List<Product>)`

Execution

When the system searches for products using `StoreAdapter.search()`, it also aims to retrieve available deals and discounts. The `ShoppingController` then uses this information to compare product prices and include applicable discounts in the shopping plan. The shopper can also view available deals and promotions through the GUI.

F08 — Alternative Product Recommendations

Related Use Case: UC05 — Find Product Alternative

Related Sequence Diagram: SD02 — Find Product Alternative

Classes Involved

- ShoppingGUI — Allows the shopper to request and view alternative products.
- ShoppingController — Receives the alternative product request.
- ShoppingAgent — Searches for and evaluates suitable substitutes.
- StoreAdapter — Retrieves available alternative products from store data sources.
- ClaudeAPI — Evaluates available alternatives and recommends a suitable product.
- Product — Represents the available products and recommended alternative.

Important Methods

- `ShoppingAgent.findAlternative(item: ShoppingItem): Product`
- `StoreAdapter.search(query: String): List<Product>`
- `ClaudeAPI.ask(prompt: String): String`
- `ShoppingGUI.showProducts(products: List<Product>)`

Execution

When the shopper requests an alternative product through the GUI, the ShoppingController uses ShoppingAgent.findAlternative() to find other options. The shopping agent searches for similar products using StoreAdapter.search() and uses ClaudeAPI.ask() to recommend a suitable alternative. If an alternative is found, it is returned to the controller and displayed through ShoppingGUI.showProducts(). Otherwise, the system informs the shopper that no suitable alternative is available.

F09 — Shopping Optimization

Related Use Case: UC06 — Optimize Shopping Plan

Related Sequence Diagram: SD03 — Optimize Shopping Plan

Classes Involved

- ShoppingGUI — Allows the shopper to select an optimization strategy and displays the result.
- ShoppingController — Coordinates the optimization process and selects the optimization implementation.
- OptimizationStrat — Defines the common interface for optimization algorithms.
- CheapestStrat — Optimizes the shopping plan to minimize the estimated cost.
- FewestStoresStrat — Optimizes the shopping plan to minimize the number of stores required.
- OneStoreStrat — Attempts to select products from a single store.
- ShoppingPlan — Represents the shopping plan being optimized.

Important Methods

- `ShoppingController.optimize(plan: ShoppingPlan): ShoppingPlan`
- `OptimizationStrat.optimize(plan: ShoppingPlan): ShoppingPlan`
- `CheapestStrat.optimize(plan: ShoppingPlan): ShoppingPlan`
- `FewestStoresStrat.optimize(plan: ShoppingPlan): ShoppingPlan`
- `OneStoreStrat.optimize(plan: ShoppingPlan): ShoppingPlan`
- `ShoppingGUI.showPlan(plan: ShoppingPlan)`

Execution

When the shopper selects an optimization option through the GUI, ShoppingController.optimize() uses the selected OptimizationStrat to improve the shopping plan. Depending on the shopper's choice, CheapestStrat finds the lowest-cost option, FewestStoresStrat reduces the number of stores to visit, or OneStoreStrat tries to find all items from one store. The optimized ShoppingPlan is then returned to the controller and displayed through ShoppingGUI.showPlan().

F10 — Interactive Checklists

Related Use Case: UC07 — Manage Shopping Checklist

Related Sequence Diagram: SD04 — Manage Shopping Checklist

Classes Involved

- ShoppingGUI — Allows the shopper to perform checklist actions and displays updated information.
- ShoppingController — Receives checklist commands and coordinates their execution.

- ChecklistCommand — Defines the common interface for executing checklist actions.
- ShoppingPlan — Contains the shopping items and manages changes to the checklist.
- ShoppingItem — Represents individual shopping items and their purchased status.
- PlanObserver — Defines the interface for receiving shopping plan updates.

Important Methods

- `ShoppingController.updateChecklist(cmd: ChecklistCommand)`
- `ChecklistCommand.execute()`
- `ShoppingPlan.addItem(item: ShoppingItem)`
- `ShoppingPlan.removeItem(item: ShoppingItem)`
- `ShoppingItem.markPurchased()`
- `ShoppingPlan.notifyObservers()`
- `PlanObserver.update(plan: ShoppingPlan)`

Execution

When the shopper makes changes to the checklist through the GUI, such as adding, removing, or marking an item as purchased, `ShoppingController.updateChecklist()` handles the request. The controller uses `ChecklistCommand.execute()` to make the selected changes to the shopping plan or item. After the checklist is updated, `ShoppingPlan.notifyObservers()` notifies the GUI to automatically display the changes.

F11 — Saved Plans and Prompts

Related Use Case: UC08 — Save/Load Shopping Plan

Related Sequence Diagram: SD05 — Save/Load Shopping Plan and Manage User Preferences

Classes Involved

- ShoppingGUI — Allows the shopper to save or reopen a shopping plan.
- ShoppingController — Handles requests to save and load shopping plans.
- ShoppingPlan — Represents the shopping plan being saved or retrieved.

Important Methods

- `ShoppingController.savePlan(plan: ShoppingPlan)`
- `ShoppingController.loadPlan(): ShoppingPlan`
- `ShoppingGUI.showPlan(plan: ShoppingPlan)`

Execution

When the shopper selects **Save Plan** through the GUI, `ShoppingController.savePlan()` saves the shopping plan and confirms that it has been saved. When the shopper selects **Load Plan**, `ShoppingController.loadPlan()` retrieves the previously saved `ShoppingPlan`. The saved plan is then returned to the GUI and displayed using `ShoppingGUI.showPlan()`.

F12 — User Preference Memory

Related Use Case: UC09 — Manage User Preferences

Related Sequence Diagram: SD05 — Save/Load Shopping Plan and Manage User Preferences

Classes Involved

- ShoppingGUI — Allows the shopper to modify their shopping preferences.
- ShoppingController — Coordinates preference updates.
- UserPreferences — Stores preferred stores and shopping restrictions.
- ShoppingAgent — Reads stored preferences when generating recommendations and shopping plans.

Important Methods

- `UserPreferences.update()`
- `ShoppingAgent.buildPlan(items: List<ShoppingItem>): ShoppingPlan`

Execution

When the shopper updates their preferences through the GUI, the ShoppingController uses `UserPreferences.update()` to save the changes. These preferences include preferred stores and shopping restrictions. The ShoppingAgent can then use these preferences when creating shopping plans or recommending products. Once the preferences are updated, the controller sends a confirmation to the GUI to inform the shopper that the changes were successful.